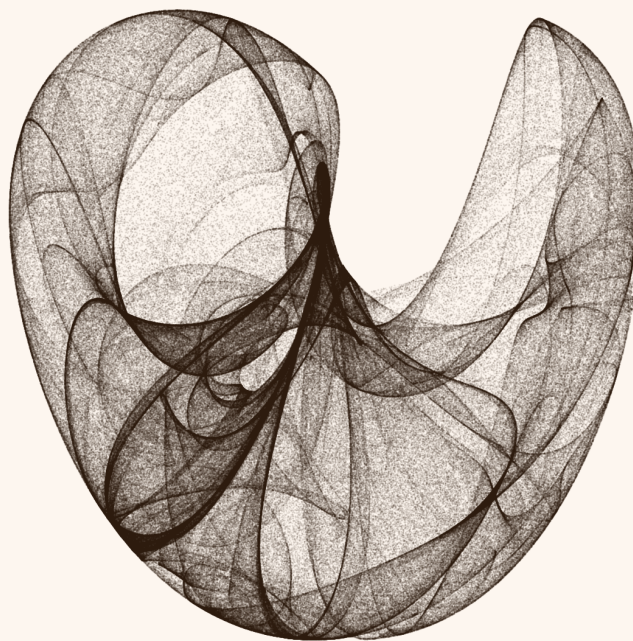




Introdução à Modelos Generativos para Imagem

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 1, 2026

Introdução à Modelos Generativos para Imagem

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 1, 2026

Contents

<i>Style Transfer</i>	4
<i>Representação do Conteúdo</i>	4
<i>Representação do Estilo</i>	5
<i>Otimização</i>	6
<i>Autoencoders</i>	6
<i>Espaço Latente</i>	7
<i>Funcionamento do Autoencoder</i>	7
<i>Função de Perda</i>	8
<i>Variações de Autoencoders</i>	8
<i>Variational Autoencoders (VAEs)</i>	9
<i>Fundamentos Probabilísticos</i>	9
<i>Encoder e Decoder Probabilísticos</i>	9
<i>Reparametrização</i>	10
<i>Função de Perda</i>	10
<i>Exemplo de Arquitetura</i>	10
<i>Vantagens e Desvantagens</i>	11
<i>Generative Adversarial Networks (GANs)</i>	11
<i>Fundamentos das GANs</i>	11
<i>Funcionamento do Treinamento</i>	12
<i>Funções de Perda</i>	12
<i>Comparação com Outras Técnicas</i>	14
<i>Modelos de Difusão</i>	14
<i>Visão Geral do Processo de Difusão</i>	14
<i>Fase de Difusão (Forward Process)</i>	14
<i>Fase de Reversão (Reverse Process)</i>	15
<i>Função de Perda</i>	15
<i>Vantagens dos Modelos de Difusão</i>	16
<i>Comparação com Outras Técnicas</i>	16
<i>Exemplo de Arquitetura: DDPM e U-Net</i>	17
<i>Stable Diffusion</i>	17
<i>Matrizes de Gram na Transferência de Estilo</i>	19
<i>Definição das Matrizes de Gram</i>	19
<i>Interpretação das Matrizes de Gram</i>	19
<i>Matrizes de Gram na Função de Perda de Estilo</i>	20
<i>Exemplo de Cálculo de Matriz de Gram</i>	20
<i>Significância das Matrizes de Gram na Transferência de Estilo</i>	21

<i>Interpretação de GANs via Teoria de Jogos</i>	22
<i>Jogo Minimax</i>	22
<i>Funções de Perda Individuais e Heurística de Saturação Não Decrescente</i>	23
<i>Estratégias dos Jogadores</i>	23
<i>Equilíbrio de Nash</i>	23
<i>Convergência do Treinamento</i>	25
<i>Exemplo Matemático Simplificado</i>	25

Style Transfer

A transferência de estilo é uma técnica de visão computacional que utiliza redes neurais convolucionais (CNNs) para gerar uma nova imagem que combina o conteúdo de uma imagem com o estilo artístico de outra. Essa técnica foi proposta no artigo seminal *A Neural Algorithm of Artistic Style* por Leon Gatys et al. e faz uso de redes convolucionais pré-treinadas, como a ResNet50, para extrair características visuais de diferentes níveis de abstração.

A metodologia se baseia na relação entre as múltiplas camadas de uma CNN, onde cada camada l produz um conjunto de mapas de características $F^l(I)$. Esses mapas capturam informações visuais em diferentes níveis de complexidade, desde bordas e texturas nas camadas iniciais até formas e objetos nas camadas mais profundas. Assim, a CNN pode ser vista como um extrator hierárquico de características, onde cada camada contribui para a representação da imagem I .

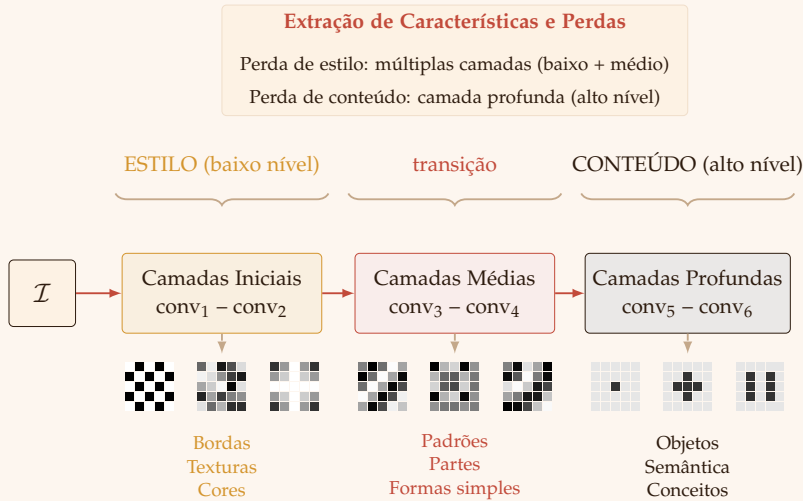


Figure 1: Hierarquia de características em uma CNN. Camadas iniciais capturam bordas, texturas e cores (estilo). Camadas médias detectam padrões e partes de objetos. Camadas profundas representam objetos completos e semântica de alto nível (conteúdo). A perda de estilo utiliza múltiplas camadas iniciais e médias, enquanto a perda de conteúdo utiliza uma camada profunda.

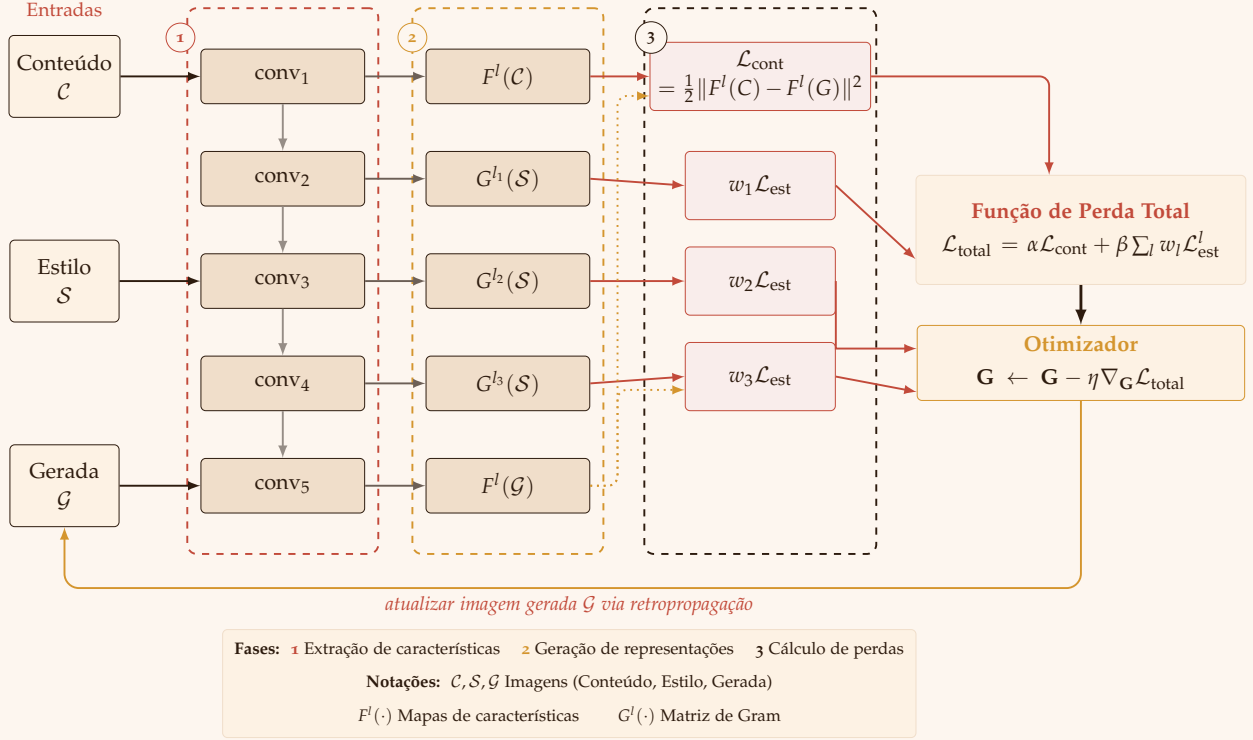
O processo de transferência de estilo depende de duas imagens de entrada:

1. **Imagem de conteúdo (C):** Contém a estrutura e os objetos principais que serão preservados na imagem gerada.
2. **Imagem de estilo (S):** Fornece as texturas, cores e padrões artísticos que serão transferidos para a imagem final.

A imagem gerada (G) é obtida por meio da otimização de uma função de perda composta, que combina a representação do conteúdo e do estilo.

Representação do Conteúdo

O conteúdo de uma imagem está associado às características de alto nível, como formas e objetos, que são capturados principalmente pelas camadas mais profundas da CNN. Para representar



o conteúdo, utilizamos os mapas de características $F^l(\mathcal{C})$ e $F^l(\mathcal{G})$ extraídos de uma camada específica l da rede.

A função de perda do conteúdo mede a diferença entre as representações da imagem de conteúdo e da imagem gerada na camada l . Essa diferença é calculada como o erro quadrático médio entre os mapas de características:

$$\mathcal{L}_{\text{conteúdo}}(\mathcal{C}, \mathcal{G}, l) = \frac{1}{2} \sum_{i,j} \left(F_{ij}^l(\mathcal{G}) - F_{ij}^l(\mathcal{C}) \right)^2$$

Onde: - $F_{ij}^l(\mathcal{C})$ é o valor do mapa de características da imagem de conteúdo na posição (i, j) da camada l . - $F_{ij}^l(\mathcal{G})$ é o valor correspondente na imagem gerada.

Representação do Estilo

O estilo de uma imagem está associado às características de baixo e médio nível, como texturas, cores e padrões, que são capturados pelas correlações entre os mapas de características em **múltiplas camadas** L da CNN. Para representar o estilo, utilizamos a matriz de Gram G^l , que mede as correlações entre os mapas de características na camada l .

A função de perda do estilo é definida como a soma ponderada das diferenças entre as matrizes de Gram da imagem de estilo e da imagem gerada em várias camadas:

Figure 2: Pipeline completo de Style Transfer. Três imagens ($\mathcal{C}$, $\mathcal{S}$, $\mathcal{G}$) passam por uma CNN pré-treinada. Mapas de características de $\mathcal{C}$ e $\mathcal{G}$ em uma camada profunda definem a perda de conteúdo. Matrizes de Gram de $\mathcal{S}$ e $\mathcal{G}$ em múltiplos níveis definem a perda de estilo. A perda total é minimizada via gradiente descendente, atualizando $\mathcal{G}$.

$$\mathcal{L}_{\text{estilo}}(S, G, L) = \sum_{l \in L} w_l \cdot \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l(G) - G_{ij}^l(S) \right)^2$$

Onde: - w_l é o peso associado à camada l , que controla sua contribuição para a perda total. - N_l é o número de mapas de características na camada l . - M_l é o tamanho espacial (altura $\times$ largura) dos mapas de características na camada l . - $G_{ij}^l(S)$ e $G_{ij}^l(G)$ são os elementos das matrizes de Gram para a imagem de estilo e a imagem gerada, respectivamente.

Otimização

A imagem gerada G é obtida minimizando uma função de perda total, que combina as perdas de conteúdo e estilo:

$$\mathcal{L}_{\text{total}}(C, S, G) = \alpha \cdot \mathcal{L}_{\text{conteúdo}}(C, G, l) + \beta \cdot \mathcal{L}_{\text{estilo}}(S, G, L)$$

Onde α e β são hiperparâmetros que controlam o equilíbrio entre a preservação do conteúdo e a transferência do estilo.

Por meio de técnicas de otimização, como o gradiente descendente, a imagem gerada é iterativamente ajustada até que a perda total seja minimizada, resultando em uma imagem que combina o conteúdo de C com o estilo de S .

O objetivo é minimizar $\mathcal{L}_{\text{total}}$ em relação à imagem gerada G . Isso é feito usando Gradient Descent:

$$G \leftarrow G - \eta \cdot \nabla_G \mathcal{L}_{\text{total}}$$

Onde η é a taxa de aprendizado.

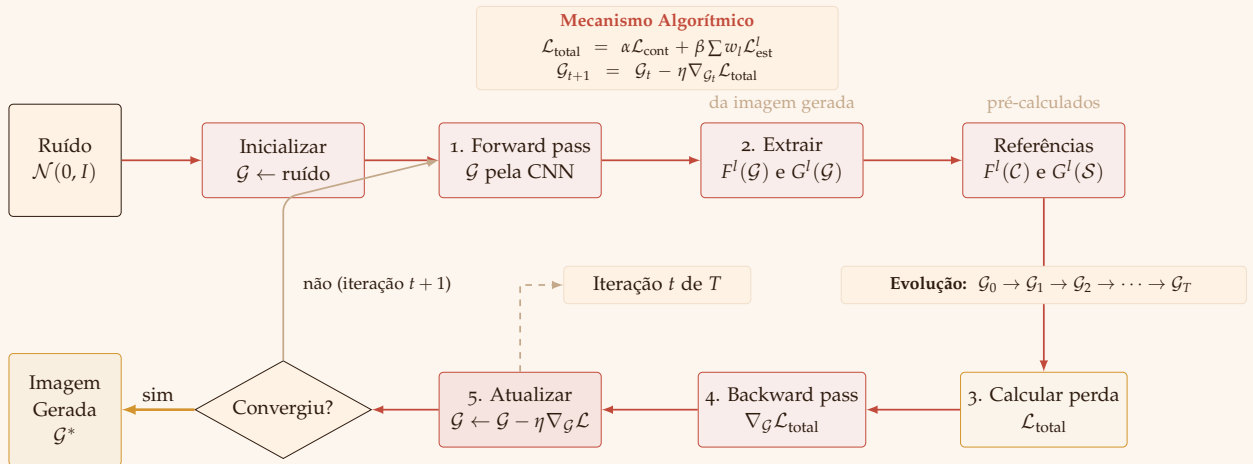


Figure 3: Loop de otimização iterativo do Style Transfer organizado em anel de fluxo horário.

Autoencoders

Autoencoders (AEs) são uma técnica de aprendizado não supervisionado que visa aprender representações compactas e eficientes

de dados sem a necessidade de rótulos. Eles consistem em duas redes neurais principais: o **encoder** (codificador) e o **decoder** (decodificador). O encoder transforma os dados de entrada em uma representação codificada, enquanto o decoder tenta reconstruir os dados originais a partir dessa representação. Matematicamente, um autoencoder pode ser definido como:

$$E_\phi : \mathcal{X} \rightarrow \mathcal{Z} \quad \text{e} \quad D_\theta : \mathcal{Z} \rightarrow \mathcal{X}$$

Onde:

- E_ϕ é o encoder, parametrizado por ϕ .
- D_θ é o decoder, parametrizado por θ .
- $\mathcal{X}$ é o espaço dos dados de entrada (por exemplo, $\mathbb{R}^m$).
- $\mathcal{Z}$ é o espaço latente (ou espaço codificado), geralmente de dimensão menor que $\mathcal{X}$ (por exemplo, $\mathbb{R}^n$, com $n \leq m$).

O objetivo do autoencoder é aprender uma representação compacta $z \in \mathcal{Z}$ dos dados de entrada $x \in \mathcal{X}$, de modo que a reconstrução $\hat{x} = D_\theta(E_\phi(x))$ seja o mais próxima possível de x .

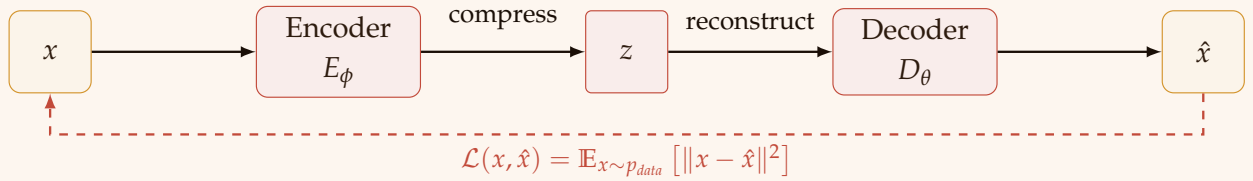


Figure 4: Exemplo clássico da arquitetura de um Autoencoder (AE).

Espaço Latente

O espaço latente $\mathcal{Z}$ é uma representação de dimensionalidade reduzida dos dados de entrada. Para cada dado $x \in \mathcal{X}$, o encoder mapeia x para um vetor latente $z \in \mathcal{Z}$:

$$z = E_\phi(x)$$

Esse vetor latente z captura as características essenciais dos dados de entrada, eliminando redundâncias e ruídos. A dimensionalidade de $\mathcal{Z}$ é tipicamente menor que a de $\mathcal{X}$, o que permite ao autoencoder aprender uma representação compacta e eficiente.

Funcionamento do Autoencoder

A intuição por trás de um autoencoder pode ser resumida em três etapas principais:

1. **Entrada:** Um dado $x \in \mathbb{R}^D$ é fornecido como entrada.

2. **Encoder:** O encoder mapeia x para um vetor latente $z \in \mathbb{R}^d$, onde $d \ll D$:

$$z = E(x) = \sigma(W_e x + b_e)$$

Aqui, W_e e b_e são os pesos e vieses do encoder, respectivamente, e σ é uma função de ativação não linear (como ReLU ou sigmoide).

3. **Decoder** O decoder reconstrói $\hat{x} \in \mathbb{R}^D$ a partir do vetor latente z :

$$\hat{x} = D(z) = \sigma(W_d z + b_d)$$

Onde W_d e b_d são os pesos e vieses do decoder.

Função de Perda

O objetivo do autoencoder é minimizar o erro de reconstrução, ou seja, a diferença entre a entrada original x e a reconstrução $\hat{x}$. A função de perda mais comum é o erro quadrático médio (MSE), definido como:

$$\mathcal{L}_{AE} = \mathbb{E}_{x \sim p_{\text{data}}} [\|x - \hat{x}\|^2]$$

Onde:

- $\mathbb{E}_{x \sim p_{\text{data}}}$ denota o valor esperado em relação à distribuição dos dados de entrada.
- $\|x - \hat{x}\|^2$ é a norma euclidiana ao quadrado da diferença entre x e $\hat{x}$.

Ao minimizar essa função de perda, o autoencoder aprende a reconstruir os dados de entrada com a maior precisão possível, ao mesmo tempo em que compacta a informação no espaço latente.

Variações de Autoencoders

Além do autoencoder básico, existem várias variações que ampliam suas capacidades:

- **Denoising Autoencoder:** Treinado para reconstruir dados a partir de entradas corrompidas por ruído.
- **Sparse Autoencoder:** Introduz uma penalidade de esparsidade no espaço latente, incentivando a ativação de apenas algumas unidades latentes.
- **Variational Autoencoder (VAE):** Modela a distribuição dos dados no espaço latente, permitindo a geração de novos dados.
- **Convolutional Autoencoder:** Utiliza camadas convolucionais no encoder e decoder, sendo especialmente útil para dados imagéticos.

Variational Autoencoders (VAEs)

Os Variational Autoencoders (VAEs) são uma extensão dos autoencoders tradicionais que incorporam conceitos de inferência probabilística. Enquanto autoencoders comuns aprendem uma representação determinística no espaço latente, os VAEs modelam a distribuição de probabilidade dos dados no espaço latente, permitindo a geração de novos dados a partir dessa distribuição.

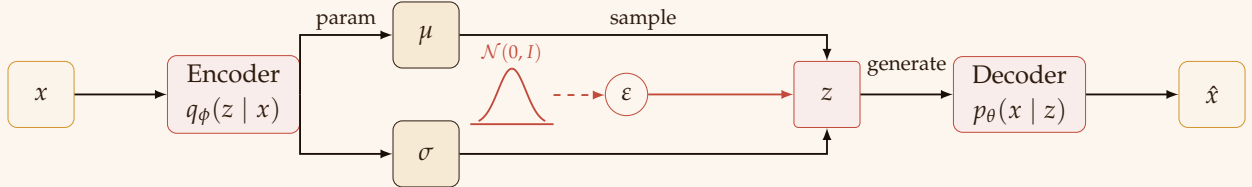


Figure 5: Arquitetura de um Variational Autoencoder (VAE) detalhando o reparameterization trick em formato expandido com amostragem exógena de uma distribuição $\mathcal{N}(0, I)$.

Fundamentos Probabilísticos

Ao contrário dos autoencoders tradicionais, que mapeiam diretamente uma entrada x para um vetor latente z , os VAEs modelam a distribuição de probabilidade $p(z|x)$. No entanto, como essa distribuição é intratável, os VAEs utilizam uma abordagem variacional para aproximá-la. Especificamente, eles introduzem uma distribuição aproximada $q_\phi(z|x)$, parametrizada por ϕ , que é aprendida durante o treinamento.

O objetivo do VAE é maximizar a ELBO, que é uma aproximação do logaritmo da verossimilhança dos dados. A ELBO é dada por:

$$\text{ELBO} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \| p(z))$$

Onde:

- $\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]$ é o termo de reconstrução, que mede quão bem o decoder consegue reconstruir x a partir de z .
- $D_{\text{KL}}(q_\phi(z|x) \| p(z))$ é a divergência de Kullback-Leibler (KL) entre a distribuição aproximada $q_\phi(z|x)$ e uma distribuição pré-definida $p(z)$ (geralmente uma distribuição normal padrão $\mathcal{N}(0, I)$).

Encoder e Decoder Probabilísticos

No VAE, tanto o encoder quanto o decoder são modelados probabilisticamente:

1. **Encoder:** O encoder mapeia a entrada x para os parâmetros de uma distribuição no espaço latente, geralmente uma distribuição normal multivariada:

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x))$$

Onde $\mu_\phi(x)$ e $\sigma_\phi^2(x)$ são a média e a variância da distribuição, respectivamente, aprendidas pelo encoder.

2. **Decoder:** O decoder mapeia o vetor latente z para os parâmetros de uma distribuição nos dados de saída. Para dados contínuos, isso geralmente é uma distribuição normal:

$$p_{\theta}(x|z) = \mathcal{N}(x; \mu_{\theta}(z), \sigma_{\theta}^2(z))$$

Para dados binários, uma distribuição Bernoulli pode ser usada.

Reparametrização

Um desafio no treinamento de VAEs é a amostragem de z a partir de $q_{\phi}(z|x)$, que é uma operação estocástica e não diferenciável. Para contornar esse problema, os VAEs utilizam de reparametrização: em vez de amostrar diretamente de $q_{\phi}(z|x)$, eles amostram de uma distribuição base $\epsilon \sim \mathcal{N}(0, I)$ e transformam a amostra da seguinte forma:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \cdot \epsilon$$

Essa transformação torna o processo diferenciável, permitindo o uso de gradientes descendentes para otimizar os parâmetros do modelo.

Função de Perda

A função de perda do VAE é o negativo da ELBO, que consiste em dois termos:

1. Termo de reconstrução: Mede a qualidade da reconstrução dos dados de entrada.
2. Termo de regularização: A divergência KL, que força a distribuição aproximada $q_{\phi}(z|x)$ a se aproximar da distribuição pré-definida $p(z)$.

Matematicamente, a função de perda é dada por:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)] + D_{\text{KL}}(q_{\phi}(z|x) || p(z))$$

Exemplo de Arquitetura

Um VAE típico consiste em:

1. Encoder: Uma rede neural que mapeia a entrada x para os parâmetros $\mu_{\phi}(x)$ e $\sigma_{\phi}(x)$ da distribuição latente.
2. Reparametrização: Amostra z usando o truque da reparametrização.
3. Decoder: Uma rede neural que mapeia z para os parâmetros da distribuição de saída $p_{\theta}(x|z)$.

Vantagens e Desvantagens

Vantagens

- Permitem a geração de novos dados a partir de uma distribuição aprendida.
- O espaço latente é contínuo, o que facilita a interpolação e a manipulação de dados.
- São baseados em fundamentos probabilísticos sólidos.

Desvantagens

- As amostras geradas podem ser menos nítidas ou realistas em comparação com outras técnicas, como GANs (Generative Adversarial Networks).
- O treinamento pode ser mais complexo devido à necessidade de otimizar a ELBO.

Generative Adversarial Networks (GANs)

As Generative Adversarial Networks (GANs) são uma classe de modelos de aprendizado profundo introduzidos por Ian Goodfellow e colaboradores em 2014. Elas são amplamente utilizadas para geração de dados, como imagens, áudio e texto. A ideia central das GANs é treinar dois modelos neurais simultaneamente: um **gerador** (generator) e um **discriminador** (discriminator), que competem entre si em um jogo de soma zero. Essa competição leva à geração de dados realistas que são indistinguíveis dos dados reais.

Fundamentos das GANs

Uma GAN consiste em dois componentes principais:

1. Gerador (G): Um modelo que gera dados a partir de um vetor de ruído aleatório z , geralmente amostrado de uma distribuição simples, como uma distribuição normal ou uniforme. O objetivo do gerador é produzir dados que sejam indistinguíveis dos dados reais.
2. Discriminador (D): Um modelo que tenta distinguir entre dados reais (amostrados da distribuição real $p_{\text{data}}(x)$) e dados falsos (gerados pelo gerador). O discriminador é treinado para maximizar a probabilidade de classificar corretamente os dados como reais ou falsos.

O treinamento das GANs é formulado como um jogo minimax entre o gerador e o discriminador, onde o gerador tenta enganar o discriminador, e o discriminador tenta detectar as amostras falsas. Matematicamente, o objetivo pode ser expresso como:

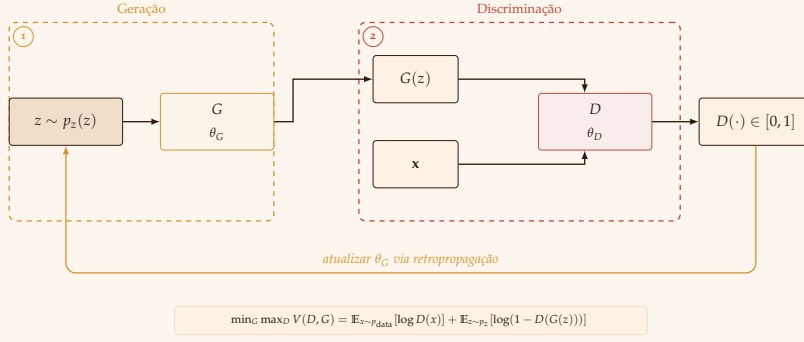


Figure 6: Arquitetura de uma GAN. Jogo adversarial de soma zero entre o gerador G e o discriminador D .

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Onde:

- $D(x)$ é a probabilidade estimada pelo discriminador de que x seja real.
- $G(z)$ é a amostra gerada pelo gerador a partir do vetor de ruído z .
- $p_{\text{data}}(x)$ é a distribuição dos dados reais.
- $p_z(z)$ é a distribuição do vetor de ruído (geralmente $\mathcal{N}(0, I)$ ou $\text{Uniform}(-1, 1)$).

Funcionamento do Treinamento

O treinamento de uma GAN envolve as seguintes etapas iterativas:

1. Treinamento do Discriminador: Fixa-se o gerador e atualiza-se o discriminador para maximizar a função de custo $V(D, G)$. Isso envolve:
 - Classificar corretamente os dados reais como reais.
 - Classificar corretamente os dados falsos (gerados pelo gerador) como falsos.
2. Treinamento do Gerador: Fixa-se o discriminador e atualiza-se o gerador para minimizar a função de custo $V(D, G)$. Isso envolve:
 - Gerar dados que sejam classificados como reais pelo discriminador.

Esse processo é repetido até que o gerador produza dados realistas e o discriminador não consiga distinguir entre dados reais e falsos.

Funções de Perda

A função de perda das GANs pode ser interpretada como uma medida de divergência entre a distribuição real $p_{\text{data}}(x)$ e a distribuição gerada $p_g(x)$. No entanto, o treinamento das GANs é

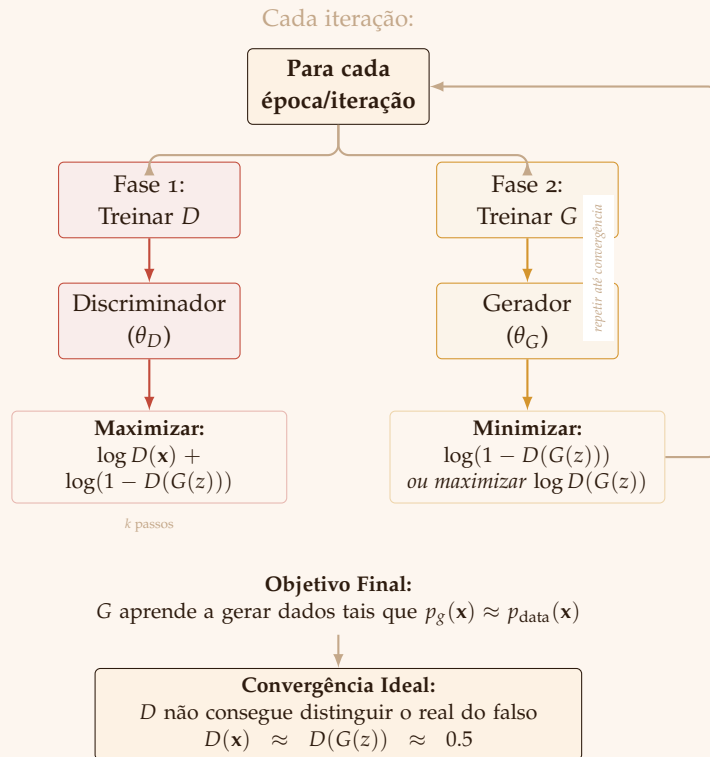


Figure 7: Ciclo de treinamento de uma GAN. O processo alterna iterativamente entre a otimização do Discriminador D (Fase 1) e do Gerador G (Fase 2) até que o sistema atinja o equilíbrio de Nash, onde as amostras sintéticas tornam-se indistinguíveis das reais.

conhecido por ser instável, o que levou ao desenvolvimento de várias variações e funções de perda alternativas, como:

- GAN Padrão: Usa a função de perda binária cruzada (binary cross-entropy).
- Wasserstein GAN (WGAN): Utiliza a distância de Wasserstein para melhorar a estabilidade do treinamento.
- Least Squares GAN (LSGAN): Substitui a função de perda binária cruzada por uma função de perda quadrática.

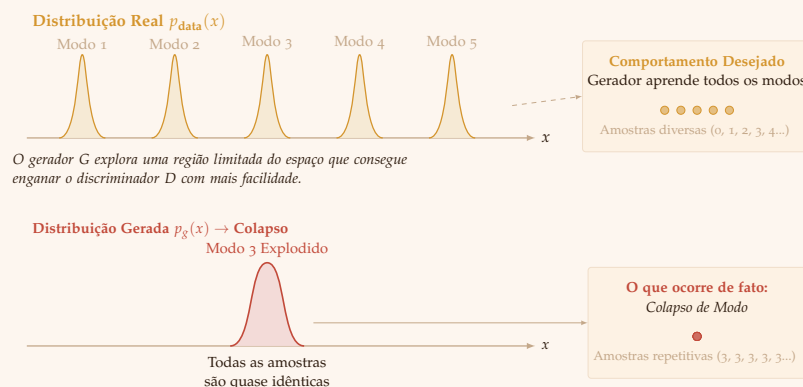


Figure 8: Colapso de Modo em GANs. Enquanto a distribuição real p_{data} apresenta múltiplos modos (alta diversidade), o gerador colapsa para uma região restrita, limitando-se a reproduzir variações de um único modo.

Comparação com Outras Técnicas

As GANs são frequentemente comparadas com outras técnicas de geração de dados, como Variational Autoencoders (VAEs):

- GANs: Produzem amostras de alta qualidade, mas são mais difíceis de treinar e avaliar.
- VAEs: São mais estáveis e fornecem uma representação probabilística do espaço latente, mas as amostras geradas podem ser menos nítidas.

Modelos de Difusão

Os Modelos de Difusão (Diffusion Models) são uma classe de modelos generativos que se baseiam em um processo de difusão para transformar dados reais em ruído e, em seguida, aprender a reverter esse processo para gerar novos dados. Eles têm se destacado por sua capacidade de produzir amostras de alta qualidade e por sua estabilidade de treinamento, superando desafios comuns em outras abordagens, como GANs (Generative Adversarial Networks) e VAEs (Variational Autoencoders).

Visão Geral do Processo de Difusão

O conceito central dos Modelos de Difusão é simular um processo de difusão, que gradualmente adiciona ruído aos dados reais até que eles se tornem indistinguíveis de ruído puro. Em seguida, o modelo aprende a reverter esse processo, transformando o ruído de volta em dados realistas. Esse processo é dividido em duas fases principais:

1. **Fase de Difusão (Forward Process):** Adiciona ruído aos dados reais em várias etapas, seguindo um esquema pré-definido.
2. **Fase de Reversão (Reverse Process):** Aprende a remover o ruído e reconstruir os dados originais.

Fase de Difusão (Forward Process)

Na fase de difusão, os dados reais $\mathbf{x}_0 \sim q(\mathbf{x})$ são gradualmente corrompidos por ruído gaussiano em T etapas. Em cada etapa t , o dado $\mathbf{x}_t$ é obtido a partir de $\mathbf{x}_{t-1}$ pela adição de ruído:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

Onde:

- β_t é um parâmetro que controla a quantidade de ruído adicionado na etapa t , geralmente definido por uma *schedule* de variâncias.
- $\mathbf{x}_{t-1}$ é a amostra da etapa anterior.

- $\mathbf{I}$ é a matriz identidade.

Definindo $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, podemos obter diretamente $\mathbf{x}_t$ a partir de $\mathbf{x}_0$ em uma única etapa:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

Isso significa que podemos amostrar $\mathbf{x}_t$ da seguinte forma:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \text{onde } \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

No final do processo de difusão ($t = T$), o dado $\mathbf{x}_T$ é essencialmente ruído gaussiano puro, sem qualquer informação discernível sobre $\mathbf{x}_0$, dado que a *schedule* de variâncias β_t é bem definida.

Fase de Reversão (Reverse Process)

A fase de reversão consiste em aprender a remover o ruído e reconstruir os dados originais. Isso é feito por meio de um modelo neural parametrizado por θ que aprende a prever a média e a variância da distribuição gaussiana que reverte o processo de difusão em cada etapa.

O processo de reversão é modelado como uma cadeia de Markov com transições gaussianas aprendidas:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Onde:

- $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ é a média prevista pelo modelo neural.
- $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$ é a variância prevista pelo modelo neural. Na prática, muitas vezes é fixada como $\sigma_t^2 \mathbf{I}$, onde σ_t^2 pode ser igual a β_t ou $\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$.

Em vez de prever diretamente $\mathbf{x}_{t-1}$, o modelo geralmente é treinado para prever o ruído ϵ adicionado em cada etapa. Assim, a média $\boldsymbol{\mu}_\theta$ pode ser reparametrizada como:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

Portanto, o processo de amostragem para gerar $\mathbf{x}_{t-1}$ a partir de $\mathbf{x}_t$ é:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad \text{onde } \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Função de Perda

O treinamento dos Modelos de Difusão é baseado na maximização da verossimilhança dos dados de treinamento. No entanto, a verossimilhança exata é intratável. Em vez disso, os modelos de difusão são treinados otimizando um limite inferior variacional (ELBO) na log-verossimilhança negativa.

Uma simplificação comum da função de perda é:

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

Onde:

- t é a etapa de difusão, amostrada uniformemente entre 1 e T .
- $\mathbf{x}_0$ é o dado real.
- $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ é o ruído adicionado durante a fase de difusão.
- $\epsilon_\theta(\mathbf{x}_t, t)$ é o ruído previsto pelo modelo, onde $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$.

Essa função de perda simplesmente compara o ruído real adicionado durante o processo de difusão com o ruído previsto pelo modelo.

Vantagens dos Modelos de Difusão

Os Modelos de Difusão apresentam várias vantagens em relação a outras técnicas generativas:

- **Estabilidade de Treinamento:** Ao contrário das GANs, que podem sofrer com instabilidade e colapso de moda, os Modelos de Difusão são mais estáveis e previsíveis.
- **Qualidade das Amostras:** Eles são capazes de gerar amostras de alta qualidade, especialmente em tarefas como geração de imagens.
- **Flexibilidade:** Podem ser aplicados a diferentes tipos de dados, como imagens, áudio e vídeos.
- **Fundamentação Teórica:** Baseiam-se em um processo bem definido de difusão e reversão, o que facilita a análise e a interpretação.

Comparação com Outras Técnicas

Em comparação com outras abordagens generativas, como GANs e VAEs, os Modelos de Difusão oferecem:

- **Melhor Qualidade:** As amostras geradas tendem a ser mais nítidas e realistas.
- **Maior Estabilidade:** O treinamento é menos propenso a problemas como colapso de moda.
- **Custo Computacional:** O treinamento e a geração podem ser mais lentos devido ao grande número de etapas de difusão e reversão.

Exemplo de Arquitetura: DDPM e U-Net

Um exemplo popular de arquitetura para Modelos de Difusão é o Denoising Diffusion Probabilistic Models (DDPM), que utiliza uma rede neural U-Net para prever o ruído $\epsilon_\theta(\mathbf{x}_t, t)$ em cada etapa de reversão. A U-Net é eficaz para capturar informações em múltiplas escalas, o que é crucial para a qualidade das amostras geradas.

A arquitetura U-Net consiste em um caminho de contração (encoder) que captura o contexto e um caminho de expansão (decoder) que permite a localização precisa. A U-Net também possui conexões residuais entre o encoder e o decoder, o que ajuda a preservar os detalhes finos durante o processo de geração.

Stable Diffusion

O Stable Diffusion é uma variação dos Modelos de Difusão que opera no **espaço latente** de um autoencoder variacional (VAE) pré-treinado, em vez de operar diretamente no espaço de pixels. Isso traz algumas vantagens:

1. **Eficiência Computacional:** Trabalhar em um espaço latente de menor dimensão reduz significativamente o custo computacional do processo de difusão e reversão.
2. **Velocidade de Amostragem:** A geração de amostras se torna mais rápida, pois o processo de reversão ocorre em um espaço de menor dimensão.
3. **Qualidade Aprimorada:** O modelo pode se concentrar em capturar a semântica e a estrutura de alto nível dos dados, enquanto o VAE cuida dos detalhes de baixo nível.

Como Funciona o Stable Diffusion

1. **Treinamento do VAE:** Um VAE é treinado para comprimir imagens em um espaço latente de menor dimensão. O encoder do VAE mapeia uma imagem $\mathbf{x}$ para uma representação latente $\mathbf{z} = E(\mathbf{x})$, e o decoder reconstrói a imagem a partir da representação latente: $\mathbf{x} \approx D(\mathbf{z})$.
2. **Difusão no Espaço Latente:** O processo de difusão é aplicado às representações latentes $\mathbf{z}$ em vez das imagens originais $\mathbf{x}$. Isso significa que $\mathbf{z}_t$ é obtido a partir de $\mathbf{z}_0$ (que é a representação latente de $\mathbf{x}_0$) adicionando ruído de acordo com a *schedule* de variâncias.
3. **Modelo de Difusão Condicionado:** Um modelo de difusão, geralmente uma U-Net, é treinado para reverter o processo de difusão no espaço latente. Esse modelo pode ser condicionado a informações adicionais, como texto (para geração de imagem a partir de texto) ou máscaras de segmentação (para edição de imagens).

4. **Geração de Amostras:** Para gerar uma nova amostra, o processo de reversão é executado no espaço latente, começando com ruído gaussiano puro $\mathbf{z}_T$. Em seguida, o decoder do VAE é usado para mapear a representação latente gerada $\mathbf{z}_0$ de volta para o espaço de pixels, produzindo a imagem final $\mathbf{x} = D(\mathbf{z}_0)$.

Arquitetura do Stable Diffusion O Stable Diffusion geralmente usa uma arquitetura U-Net para o modelo de difusão no espaço latente. Além disso, ele pode incorporar mecanismos de atenção cruzada (cross-attention) para condicionar a geração a entradas de texto. Isso permite que o modelo gere imagens que correspondam a descrições textuais específicas.

Em resumo, o Stable Diffusion é uma abordagem eficiente e poderosa para a geração de imagens de alta qualidade, combinando a força dos Modelos de Difusão com a eficiência de trabalhar em um espaço latente aprendido por um VAE.

Matrizes de Gram na Transferência de Estilo

As matrizes de Gram desempenham um papel crucial na captura do estilo de uma imagem durante o processo de transferência de estilo. Elas são usadas para medir as correlações entre os mapas de características em diferentes camadas de uma rede neural convolucional (CNN). Esta seção fornece uma explicação detalhada das matrizes de Gram, seu cálculo e sua importância na transferência de estilo.

Definição das Matrizes de Gram

Dado um conjunto de mapas de características F^l extraídos de uma camada específica l de uma CNN, a matriz de Gram G^l é uma matriz quadrada que captura os produtos internos entre os mapas de características vetorizados. Se os mapas de características na camada l têm dimensões $N_l \times M_l$ (onde N_l é o número de mapas de características e M_l é a dimensão espacial, ou seja, altura $\times$ largura), a matriz de Gram G^l é definida como:

$$G^l = F^l \cdot (F^l)^T$$

Onde:

- F^l é a matriz de mapas de características vetorizados com dimensões $N_l \times M_l$.
- $(F^l)^T$ é a transposta de F^l .
- A matriz de Gram resultante G^l tem dimensões $N_l \times N_l$.

Cada elemento G_{ij}^l da matriz de Gram representa o produto interno entre o i -ésimo e o j -ésimo mapa de características:

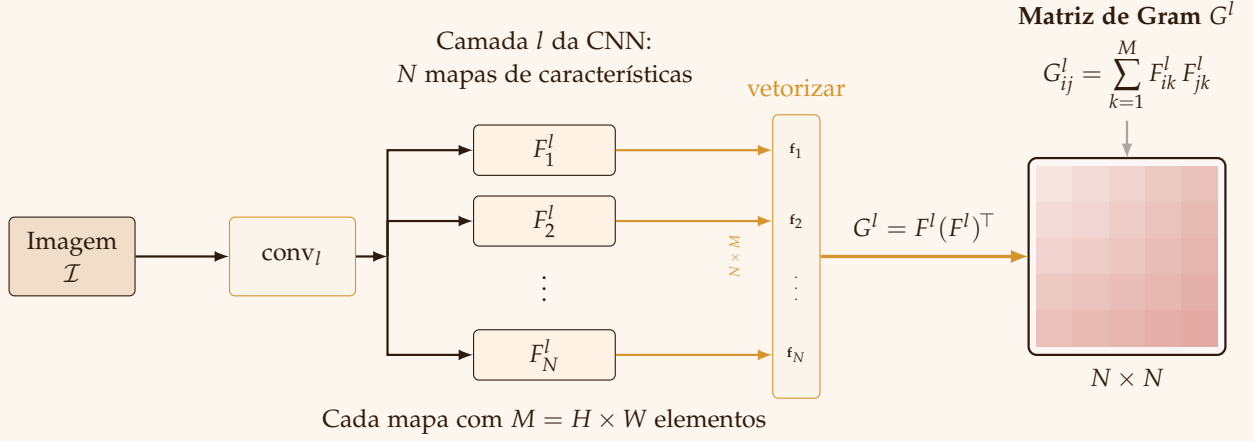
$$G_{ij}^l = \sum_{k=1}^{M_l} F_{ik}^l \cdot F_{jk}^l$$

Interpretação das Matrizes de Gram

A matriz de Gram codifica informações sobre as correlações entre os mapas de características em uma determinada camada. Essas correlações capturam a textura e o estilo de uma imagem, pois representam a frequência com que certas características (por exemplo, bordas, cores, padrões) aparecem juntas na imagem. Por exemplo:

- Valores altos na matriz de Gram indicam correlações fortes entre mapas de características específicos, o que corresponde a padrões ou texturas recorrentes na imagem.
- Valores baixos indicam correlações fracas ou inexistentes, sugerindo a ausência de tais padrões.

Ao comparar as matrizes de Gram da imagem de estilo e da imagem gerada, o algoritmo de transferência de estilo garante que a imagem gerada imite a textura e o estilo da imagem de estilo.



Matrizes de Gram na Função de Perda de Estilo

A função de perda de estilo na transferência de estilo é calculada usando as matrizes de Gram da imagem de estilo (S) e da imagem gerada (G). Para uma camada específica l , a perda de estilo é definida como o erro quadrático médio entre as matrizes de Gram das duas imagens:

$$\mathcal{L}_{\text{estilo}}^l(S, G) = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l(G) - G_{ij}^l(S) \right)^2$$

Onde:

- $G_{ij}^l(S)$ é o elemento da matriz de Gram para a imagem de estilo.
- $G_{ij}^l(G)$ é o elemento da matriz de Gram para a imagem gerada.
- N_l é o número de mapas de características na camada l .
- M_l é a dimensão espacial dos mapas de características na camada l .

A perda de estilo total é calculada como uma soma ponderada das perdas de estilo em várias camadas L :

$$\mathcal{L}_{\text{estilo}}(S, G, L) = \sum_{l \in L} w_l \cdot \mathcal{L}_{\text{estilo}}^l(S, G)$$

Onde w_l é o peso atribuído à camada l .

Exemplo de Cálculo de Matriz de Gram

Considere um exemplo simplificado em que uma camada l tem $N_l = 3$ mapas de características, cada um com dimensão espacial de $M_l = 2 \times 2 = 4$. Os mapas de características podem ser representados como:

$$F^l = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 2 & 2 \end{bmatrix}$$

Figure 9: Cálculo da Matriz de Gram na camada l . Os N mapas de características são vetorizados e organizados na matriz F^l de dimensão $N \times M$. O produto $F^l (F^l)^T$ gera a matriz de Gram G^l de tamanho $N \times N$, onde cada elemento G_{ij}^l mede a correlação entre os mapas i e j .

A matriz de Gram G^l é calculada como:

$$G^l = F^l \cdot (F^l)^T = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \\ 3 & 1 & 2 \\ 4 & 3 & 2 \end{bmatrix} = \begin{bmatrix} 30 & 20 & 20 \\ 20 & 14 & 13 \\ 20 & 13 & 14 \end{bmatrix}$$

Cada elemento de G^l representa a correlação entre os mapas de características correspondentes.

Significância das Matrizes de Gram na Transferência de Estilo

As matrizes de Gram são essenciais para a transferência de estilo porque fornecem uma maneira de quantificar e comparar a textura e o estilo das imagens. Ao minimizar a diferença entre as matrizes de Gram da imagem de estilo e da imagem gerada, o algoritmo garante que a imagem gerada adote o estilo artístico da imagem de estilo enquanto preserva o conteúdo da imagem de conteúdo.

Interpretação de GANs via Teoria de Jogos

As Generative Adversarial Networks (GANs) podem ser interpretadas como um jogo entre dois jogadores: o gerador (G) e o discriminador (D). Neste apêndice, exploramos como a teoria de jogos fornece uma estrutura matemática para entender o treinamento e a convergência das GANs, baseando-nos no artigo original de Goodfellow et al. (2014).

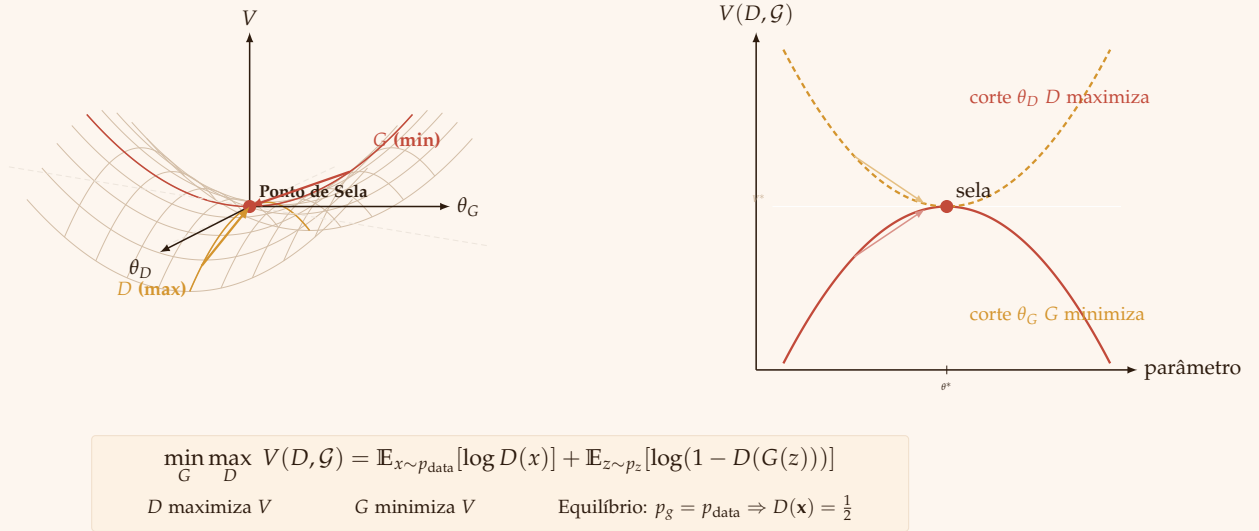
Jogo Minimax

O artigo original de Goodfellow et al. (2014) formula o treinamento de GANs como um **jogo minimax**. Se o discriminador for ótimo, este jogo se torna de soma zero. No entanto, na prática, a otimização simultânea das funções de perda do gerador e discriminador geralmente resulta em um jogo de soma não-zero. Apesar disso, o princípio fundamental de competição entre os dois jogadores permanece.

A função de valor $V(G, D)$, que define o "pagamento" do jogo, é expressa como:

$$\min_G \max_D V(G, D) = \min_{\theta_g} \max_{\theta_d} \left(\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \right)$$

Onde:



O objetivo é encontrar um ponto de sela desta função, onde o discriminador maximiza $V(G, D)$ em relação a θ_d e o gerador minimiza $V(G, D)$ em relação a θ_g .

Figure 10: Treinamento de GANs como jogo minimax de soma zero. **Esquerda:** superfície de sela $V(\theta_D, \theta_G)$ com o equilíbrio de Nash. As setas mostram a convergência — D ao longo da crista (maximização) e G ao longo do vale (minimização). **Direita:** cortes unidimensionais exibindo máximo local para D e mínimo local para G no ponto de sela.

Funções de Perda Individuais e Heurística de Saturação Não Decrescente

Embora a equação acima represente o objetivo geral, o artigo original define as funções de perda individuais do discriminador e do gerador como:

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2}\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] - \frac{1}{2}\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$$J^{(G)}(\theta_d, \theta_g) = -J^{(D)}(\theta_d, \theta_g)$$

No entanto, para lidar com problemas de saturação e gradientes fracos no início do treinamento, o artigo original também propõe uma **alternativa** à função de perda do gerador, conhecida como **heurística de saturação não decrescente**:

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2}\mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))]$$

Essa modificação fornece gradientes mais fortes para o gerador quando o discriminador rejeita facilmente as amostras geradas.

Estratégias dos Jogadores

Cada jogador (gerador e discriminador) tem uma estratégia que consiste em ajustar seus parâmetros (θ_g e θ_d , respectivamente) para otimizar sua função objetivo:

1. **Discriminador (D)**: Escolhe seus parâmetros θ_d para maximizar $J^{(D)}(\theta_d, \theta_g)$. Isso envolve classificar corretamente os dados reais como reais e os dados falsos como falsos. O discriminador é atualizado usando gradiente ascendente em sua função de perda.
2. **Gerador (G)**: Escolhe seus parâmetros θ_g para minimizar $J^{(G)}(\theta_d, \theta_g)$, seja usando a formulação original (minimizando a divergência de Jensen-Shannon¹) ou a heurística de saturação não decrescente. O gerador é atualizado usando algoritmos de otimização baseados em gradiente em sua função de perda.

O treinamento das GANs pode ser visto como um processo iterativo em que os jogadores ajustam suas estratégias em resposta às ações do oponente. O artigo original sugere atualizar o discriminador k vezes para cada atualização do gerador.

¹ A divergência de Jensen-Shannon é uma derivação da divergência de Kullback-Leibler, com diferenças sutis como o resultado ser sempre um valor finito. Sua raiz quadrada é chamada de Distância de Jensen-Shannon: a similaridade das distribuições é maior quando a distância for mais próxima à zero.

Equilíbrio de Nash

Para entender o conceito de Equilíbrio de Nash em GANs, é útil defini-lo formalmente no contexto da teoria de jogos. Considere um jogo com dois jogadores, onde:

- G é o gerador, com espaço de estratégias S_G (o conjunto de todas as possíveis distribuições que o gerador pode produzir).

- D é o discriminador, com espaço de estratégias S_D (o conjunto de todas as possíveis funções discriminadoras).
- $V(G, D)$ é a função de valor (ou payoff) que define o resultado do jogo para cada combinação de estratégias.

Definição: Um par de estratégias (G^*, D^*) é um **Equilíbrio de Nash** se, e somente se, as seguintes condições forem satisfeitas:

1. Melhor resposta do discriminador: Para qualquer estratégia alternativa do discriminador $D \in S_D$, temos:

$$V(G^*, D^*) \geq V(G^*, D)$$

Isso significa que, dado que o gerador está jogando a estratégia G^* , o discriminador não pode obter um resultado melhor do que jogar D^* . Em outras palavras, D^* é a melhor resposta do discriminador à estratégia G^* .

2. Melhor resposta do gerador: Para qualquer estratégia alternativa do gerador $G \in S_G$, temos:

$$V(G^*, D^*) \leq V(G, D^*)$$

Isso significa que, dado que o discriminador está jogando a estratégia D^* , o gerador não pode obter um resultado melhor do que jogar G^* . Em outras palavras, G^* é a melhor resposta do gerador à estratégia D^* .

Em termos matemáticos mais compactos:

Um par de estratégias (G^*, D^*) é um Equilíbrio de Nash se:

$$V(G^*, D^*) = \max_{D \in S_D} V(G^*, D) = \min_{G \in S_G} V(G, D^*)$$

No equilíbrio de Nash, nenhum jogador tem incentivo para mudar sua estratégia unilateralmente. Qualquer desvio da estratégia de equilíbrio resultará em um resultado pior para o jogador que se desviou, assumindo que o outro jogador permaneça com sua estratégia de equilíbrio.

No contexto das GANs, o equilíbrio de Nash global idealmente ocorre quando:

- O gerador G^* produz amostras da distribuição real de dados ($p_g(x) = p_{\text{data}}(x)$).
- O discriminador D^* atribui uma probabilidade de $1/2$ para todas as entradas ($D^*(x) = 1/2$ para todo x), indicando que ele não pode distinguir entre amostras reais e geradas.

Importante:

- A existência de um equilíbrio de Nash é garantida em jogos finitos (com um número finito de jogadores e estratégias) pelo Teorema de Nash. No entanto, GANs operam em espaços de estratégias contínuos e de alta dimensão, tornando a análise mais complexa.

- A definição acima refere-se ao **equilíbrio de Nash puro**. Existem também **equilíbrios de Nash mistos**, onde os jogadores escolhem suas estratégias probabilisticamente.

Conexão com a Divergência de Jensen-Shannon (Demonstrado no Artigo Original) Quando o discriminador é ótimo, **minimizar a função de valor $V(G, D)$ é equivalente a minimizar a divergência de Jensen-Shannon (JSD) entre p_{data} e p_g** , conforme a seguinte relação:

$$V(G, D_G^*) = 2 \cdot D_{JS}(p_{\text{data}} || p_g) - \log 4$$

Isso significa que, sob condições ideais, o treinamento de GANs visa minimizar a JSD, uma medida simétrica de similaridade entre distribuições.

Convergência do Treinamento

Embora o artigo original apresente uma prova teórica de convergência para um algoritmo idealizado, ele também reconhece as dificuldades práticas, como:

1. **Capacidade Limitada dos Modelos:** Redes neurais têm capacidade finita.
2. **Otimização Imperfeita:** O discriminador nem sempre atinge seu ótimo na prática.
3. **Aproximações da Função de Perda:** A heurística de saturação não decrescente, embora útil, não possui a mesma base teórica que a minimização da JSD.

Esses desafios levam a problemas bem conhecidos no treinamento de GANs:

- **Instabilidade:** A natureza antagônica do jogo pode causar oscilações e dificultar a convergência.
- **Colapso de Modo:** O gerador pode aprender a produzir apenas uma variedade limitada de amostras, perdendo a diversidade da distribuição real.

Exemplo Matemático Simplificado

Considere um caso simplificado para ilustrar o conceito de equilíbrio de Nash:

- Espaço de dados unidimensional: $x \in \mathbb{R}$.
- Distribuição real: $p_{\text{data}}(x) = \mathcal{N}(0, 1)$ (Gaussiana padrão).
- Gerador: $G(z) = \theta_g z$, onde $z \sim \mathcal{N}(0, 1)$.
- Discriminador: $D(x) = \sigma(\theta_d x)$, onde σ é a função sigmoide.

Neste cenário, o equilíbrio de Nash ocorre quando $\theta_g = 1$ e $\theta_d = 0$. Isso significa que:

- O gerador replica perfeitamente a distribuição real.
- O discriminador se torna incapaz de distinguir entre dados reais e falsos, atribuindo uma probabilidade de 0,5 para ambos.

Demonstração Para um gerador fixo, o discriminador ótimo é:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \frac{\mathcal{N}(x; 0, 1)}{\mathcal{N}(x; 0, 1) + \mathcal{N}(x; 0, \theta_g^2)}$$

No equilíbrio de Nash, $p_g(x) = p_{\text{data}}(x)$, o que implica $\theta_g = 1$. Nesse caso, $D_G^*(x) = 1/2$.

Para encontrar o θ_d ótimo, pode-se derivar a função de perda do discriminador em relação a θ_d . Substituindo $D(x)$ pela expressão do discriminador ótimo $D_G^*(x)$ e considerando o equilíbrio de Nash ($\theta_g = 1$), onde $p_g(x) = p_{\text{data}}(x)$, a derivada se simplifica e se torna zero quando $\theta_d = 0$. Isso indica que, no equilíbrio, o discriminador se torna uma função constante, atribuindo a probabilidade de 0.5 para qualquer entrada.

Este exemplo é extremamente simplificado e serve apenas para fins ilustrativos. Em casos reais, encontrar o equilíbrio de Nash analiticamente é inviável, e o treinamento de GANs depende de métodos numéricos para aproximá-lo.